

System Design Document

OutYah

Jamaica Outing Discovery Platform

Week 3 Deliverable (July 13 - July 17, 2026) - System Design

Student: Khrisean Stewart

Course: Special Projects

Date: 31 July 2026

Repository: https://github.com/KhriseanStewart/VTDI_Platform

Live app: <https://vtdi-platform.vercel.app>

1. Architecture Overview

OutYah uses a client-server architecture: a React SPA on Vercel communicates with Supabase (Auth, Postgres, Storage) and Google Maps Platform APIs. The browser owns presentation and interaction, while Supabase provides the durable data and security boundary. Context providers keep authentication, catalog data, and the user's saved outing state available across routes without tightly coupling page components.

Browser (React/Vite)

- | - AuthContext / DataContext / AppContext
- | - Public pages + Admin portal
- + -> Supabase Auth + REST (PostgREST) + Storage
- + -> Google Maps JS + Directions

2. Technology Selection

- Frontend: React 19, Vite, React Router, Lucide icons
- Backend-as-a-Service: Supabase (Postgres, Auth, RLS, Storage)
- Maps: Google Maps JavaScript API + Directions API
- Hosting: Vercel (SPA rewrites in vercel.json)
- Version control: GitHub

2.1 Selection rationale

React and Vite were selected for fast component-driven development and a lightweight production build. Supabase reduces backend setup while retaining a relational database, row-level security, authentication, and storage. Google Maps supplies reliable map tiles, markers, and routing. Vercel provides automatic deployment from GitHub and supports the single-page rewrite required by React Router.

3. Database Design (ERD description)

Core entities and relationships:

- profiles (1:1 auth.users) - role user|admin
- places - venues with lat/lng, images, hours JSON, reviews JSON
- events - optional FK to places
- posts - Instagram-style media; optional FK to places
- post_comments - FK to posts
- favorites - composite PK (user_id, place_id)
- plan_stops - composite PK (user_id, place_id) + position

auth.users --1:1-- profiles

places --1:N-- events

places --1:N-- posts --1:N-- post_comments

users --N:M-- places (via favorites)

users --N:M-- places (via plan_stops ordered)

4. Security Design (RLS)

- Public SELECT on places, events, posts, comments
- Admin ALL on catalog tables via is_admin()
- Users manage only their favorites and plan_stops
- Storage bucket media: public read, admin write

The client uses only the Supabase anon key. Authorization is enforced in PostgreSQL rather than by hiding controls in

the browser. Admin screens still check `profiles.role` for a clear user experience, but RLS remains the final enforcement layer. Database passwords and the service-role key are server-only and excluded from Git.

5. Interface Design

Public shell: desktop sidebar + mobile bottom nav. Home uses a Feed/Map segmented control. Admin uses a separate layout without public bottom navigation. Empty states provide clear CTAs when catalogs are empty.

5.1 Responsive behavior

- Desktop: persistent left navigation and multi-column content grids.
- Mobile: fixed bottom navigation, horizontally scrollable chips, and single-column forms.
- Feed cards emphasize photography with minimal borders and no heavy elevation.
- Admin tables scroll horizontally on small screens while forms collapse to one column.

5.2 Data-state behavior

Every catalog screen distinguishes loading, populated, empty, and error states. Empty states explain what is missing and provide an appropriate action, such as adding the first place for an administrator or returning to discovery for a regular user.

6. Key UML-style Flows

6.1 Discovery to Plan

1. User opens Feed, filters/search posts
2. Opens Venue Detail
3. Favorites and/or Add to outing
4. Planner shows ordered stops + route map
5. Get Directions opens Google Maps URL

6.2 Admin content publish

1. Admin signs in
2. `profiles.role` verified as admin
3. CRUD place/event/post in `/admin`
4. Public `DataContext` refresh shows content

7. Deployment Design

- Build: `bun/vite -> dist/`
- Vercel env: `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_GOOGLE_MAPS_API_KEY`
- DB script: `supabase/full_setup.sql` (schema + optional seed)
- Client never receives `service_role` key

8. Module Responsibilities

- `AuthContext`: session lifecycle, profile lookup, sign-in, sign-up, sign-out, admin role.
- `DataContext`: Supabase catalog queries, refresh, and in-memory lookup helpers.
- `AppContext`: favorites and ordered plan state, including per-user persistence.
- `lib/data.js`: database-row to UI-model mapping and Supabase query functions.
- `pages/admin`: protected dashboard and CRUD interfaces.